

Task 2

I copy pasted my lab 4 code and inserted the required code for lab 5

Here I allocate 512 words of stack memory:

```
# Interrupt stack (.space allocates bytes, so 512 words = 2048 bytes)
kstack: .space 2048
```

And here is my pseudo ISR:

Note that I used some different registers to the instructions because I was already using the ones that the instructions said to use.

First I am pretending to disable irqs, then initialise the stack pointer and push the \$t registers into the stack.

```
208 # ISR for lab 5
209 pseudoisr:
210 # PRETEND DISABLE IRQS
211
212 # Initialise stack pointer
213 la $s6, kstack
214
215 # Push contents of $t0 - $t6 to the stack
216 sw $t0, 0($s6)
217 addi $s6, $s6, 4
218 sw $t1, 0($s6)
219 addi $s6, $s6, 4
220 sw $t2, 0($s6)
221 addi $s6, $s6, 4
222 sw $t3, 0($s6)
223 addi $s6, $s6, 4
224 sw $t4, 0($s6)
225 addi $s6, $s6, 4
226 sw $t5, 0($s6)
227 addi $s6, $s6, 4
228 sw $t6, 0($s6)
229 addi $s6, $s6, 4
```

Once they have been pushed, it is safe to set them to some other values:

```
231 # Setting registers $t0 - $t6 to distinct values
232 li $t0, 5
233 li $t1, 10
234 li $t2, 15
235 li $t3, 20
236 li $t4, 25
237 li $t5, 30
238 li $t6, 35
```

Then I can compute the average by adding them up and dividing, accessing the quotient from the LO register and storing it in \$s5:

```

240 # Computing average and storing in $s5 (mflo gets the quotient from the LO register)
241 add $s5, $t0, $t1
242 add $s5, $s5, $t2
243 add $s5, $s5, $t3
244 add $s5, $s5, $t4
245 add $s5, $s5, $t5
246 add $s5, $s5, $t6
247
248 li $t0, 7
249 div $s5, $t0
250 mflo $s5

```

The average should be 20 (or 0x14 in hex), we can see this stored in \$s5:

\$s5	21	0x00000014
------	----	------------

Then the final step is to pop the values off the stack back into registers \$t0-\$t6, pretend to enable irqs again and then return to the main program:

```

252 # Popping values off the stack
253 addi $s6, $s6, -4
254 lw $t6, 0($s6)
255 addi $s6, $s6, -4
256 lw $t5, 0($s6)
257 addi $s6, $s6, -4
258 lw $t4, 0($s6)
259 addi $s6, $s6, -4
260 lw $t3, 0($s6)
261 addi $s6, $s6, -4
262 lw $t2, 0($s6)
263 addi $s6, $s6, -4
264 lw $t1, 0($s6)
265 addi $s6, $s6, -4
266 lw $t0, 0($s6)
267
268 # PRETEND ENABLE IRQS
269
270 j returnfrompseudoisr

```

Testing: Here you can see I have inserted the ISR in the middle of reading characters from the user's input:

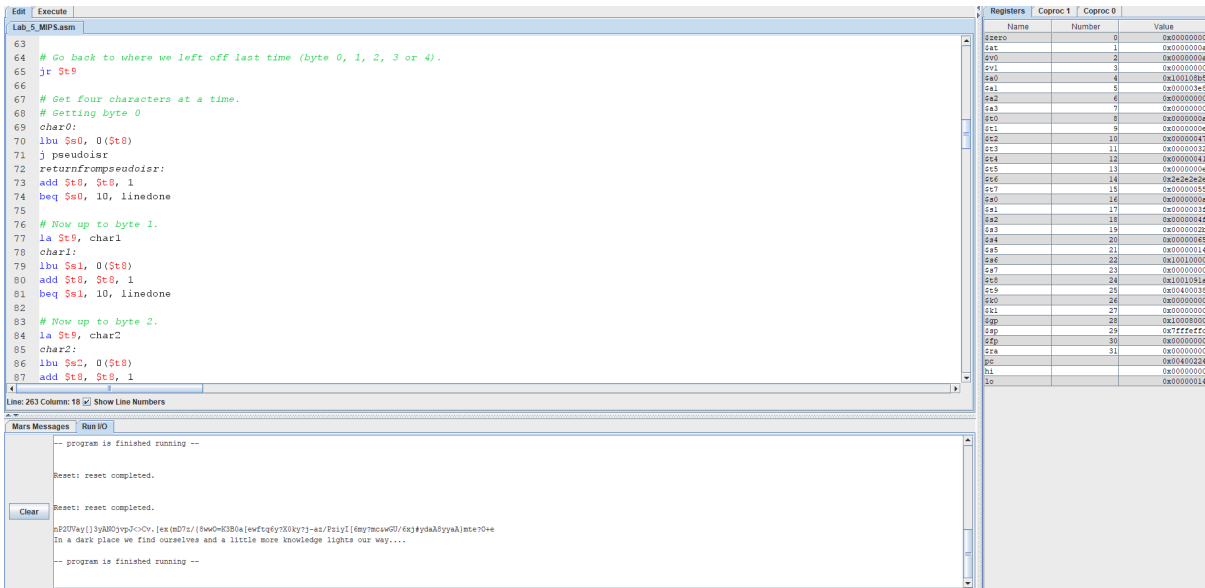
```

67 # Get four characters at a time.
68 # Getting byte 0
69 char0:
70 lbu $s0, 0($t8)
71 j pseudoisr
72 returnfrompseudoisr:
73 add $t8, $t8, 1
74 beq $s0, 10, linedone
75
76 # Now up to byte 1.
77 la $t9, char1
78 char1:
79 lbu $s1, 0($t8)
80 add $t8, $t8, 1
81 beq $s1, 10, linedone
82

```

When I run this and step through I can see the program switch to my ISR, save the \$t registers in the stack, run the ISR, reload the \$t registers and then switch back to the main program.

Here is a screenshot of the output after running the program:



You can see that the program ran all the way through, as it decoded the encoded message that I entered. On the right you can see \$s6 holding the stack pointer address, and \$s5 is holding 0x14 (the average of the \$t registers from the ISR), so we know the ISR ran throughout the program execution.

The result is the same no matter where I place the ISR, I tried 3 spots.

Task 3

In my excel sheet, I made a table of all the instruction in my ISR, and gave them a class. I used xlookup to pull their number of cycles from the other table, and then summed up the cycles.

	A	B	C	D	E	F
1	Instruction	Type	Cycles			
2						
3	j pseudoisr	jump	3			
4					Total cycles	220
5	# PRETEND DISABLE IRQS					
6						
7	# Initialise stack pointer					
8	la \$s6 kstack	load	5			
9						
10	# Push contents of \$t0 - \$t6 to the stack					
11	sw \$t0 0(\$s6)	store	4			
12	addi \$s6, \$s6, 4	simple int op	4			
13	sw \$t1 0(\$s6)	store	4			
14	addi \$s6, \$s6, 4	simple int op	4			
15	sw \$t2 0(\$s6)	store	4			

My other sheet has the lookup table, as well as the CPUs and their clock speeds:

	A	B	C	D
1		Chip	Clock freq (GHz)	Time to Run ISR (ns)
2	Case A	ARM Cortex-A5	0.4	550
3	Case B	Qualcomm Snapdragon 410	1.2	183.3333333
4	Case C	Qualcomm Snapdragon 888	2.84	77.46478873
5	Case D	Apple M2 Pro	3.78	58.2010582
6	Case E	Intel Core i9-13900K	5.3	41.50943396
7				
8		Instruction Type	Cycles	
9		load	5	
10		store	4	
11		register op	2	
12		simple int op	4	
13		complex int op	32	
14		branch	3	
15		jump	3	

All I needed to do was multiply the cycle time (1/Hz) and multiply this with the number of cycles calculated in the other sheet. Then I converted the time to run into ns to easily compare the different processors.